

AWS Multi-Region Resilience Monitor

DNS Health Detection and Regional Failover for AWS

Technical Documentation

Ritvik Indupuri

Abstract

AWS Multi-Region Resilience Monitor is a Python-based monitoring and failover prototype for AWS service availability. It resolves regional AWS service endpoints, measures DNS and connection behavior, groups failures by service and region, estimates dependent-service exposure and publishes operational telemetry. A Flask and Socket.IO dashboard supplies a local real-time view, while deployment utilities define scheduled Lambda checks, EventBridge rules, CloudWatch dashboards and alarms, an SNS topic, and regional infrastructure.

A separate failover manager evaluates Application Load Balancer, EC2, RDS and ECS health in a primary and two secondary regions. When its health and consecutive-failure thresholds are crossed, it can select a healthier region, update Route 53 records and CloudFront origins, increase Auto Scaling and ECS capacity, verify readiness, and roll back routing changes if verification fails.

Document scope

- System boundaries and the single end-to-end architecture view.
- DNS and TCP health collection across regional AWS service endpoints.
- Failure-rate analysis, cascade-risk scoring and anomaly detection.
- Regional health evaluation, failover, verification and rollback behavior.
- CloudWatch telemetry, dashboards, local visualization and deployment assets.
- Security posture, implementation cross-reference and prioritized next steps.

Contents

Abstract	2
1 System overview	3
2 System architecture	4
3 Architecture interpretation	5
4 DNS health collection	5
5 Detection and risk analysis	6
6 Multi-region health and failover	7
7 Telemetry and operational views	7
8 Infrastructure and deployment	8
9 Security and operating boundaries	8
10 Implementation cross-reference	9
11 Next steps and completion criteria	10

1 System overview

The project combines three complementary execution surfaces. The core monitor runs as a long-lived local Python process and performs detailed endpoint checks and failure analysis. The web dashboard runs its own lightweight monitor and streams current status to a browser every ten seconds. AWS deployment utilities define regional scheduled functions and CloudWatch resources for unattended telemetry. Regional failover is implemented by a dedicated manager rather than by the DNS analysis class itself.

The architecture distinguishes implemented code paths from their deployment packaging. This keeps the behavior of the core monitor, scheduled functions and failover manager precise without presenting them as a single production service where the repository does not yet wire them together.

1.1 Implemented capabilities

Capability	Implemented behavior
Endpoint observation	DNS A-record resolution for DynamoDB, RDS, Lambda, EC2, ELB and S3 endpoints across three default regions; the core monitor also attempts TCP connectivity on port 443.
Failure analysis	Groups checks by service and region, calculates failure rates, creates severity-aware alerts and estimates cascade exposure using a dependency map.
Anomaly detection	Fits an Isolation Forest over success, response time and hour-of-day features when sufficient check history exists.
Regional resilience	Scores ELB, EC2, RDS and ECS health, chooses a secondary region, updates DNS/CDN routing, scales target capacity, verifies health and rolls back routing on failure.
Operations	Publishes custom CloudWatch metrics, creates dashboards and alarms, exposes local web status, and defines scheduled EventBridge and Lambda resources.

1.2 Technology profile

Layer	Technology
Runtime	Python, boto3, dnspython, sockets and background threads
Analysis	NumPy, pandas, scikit-learn Isolation Forest and StandardScaler
Web	Flask, Flask-SocketIO and a standalone HTML dashboard
AWS	Lambda, EventBridge, CloudWatch, Route 53, CloudFront, Auto Scaling, ECS, ELB, EC2, RDS and SNS
Infrastructure	CloudFormation YAML plus Python deployment orchestration

2 System architecture

The system has two monitoring entry paths and a separate regional resilience controller. Official AWS service icons identify managed services; unbranded boxes identify project Python components.

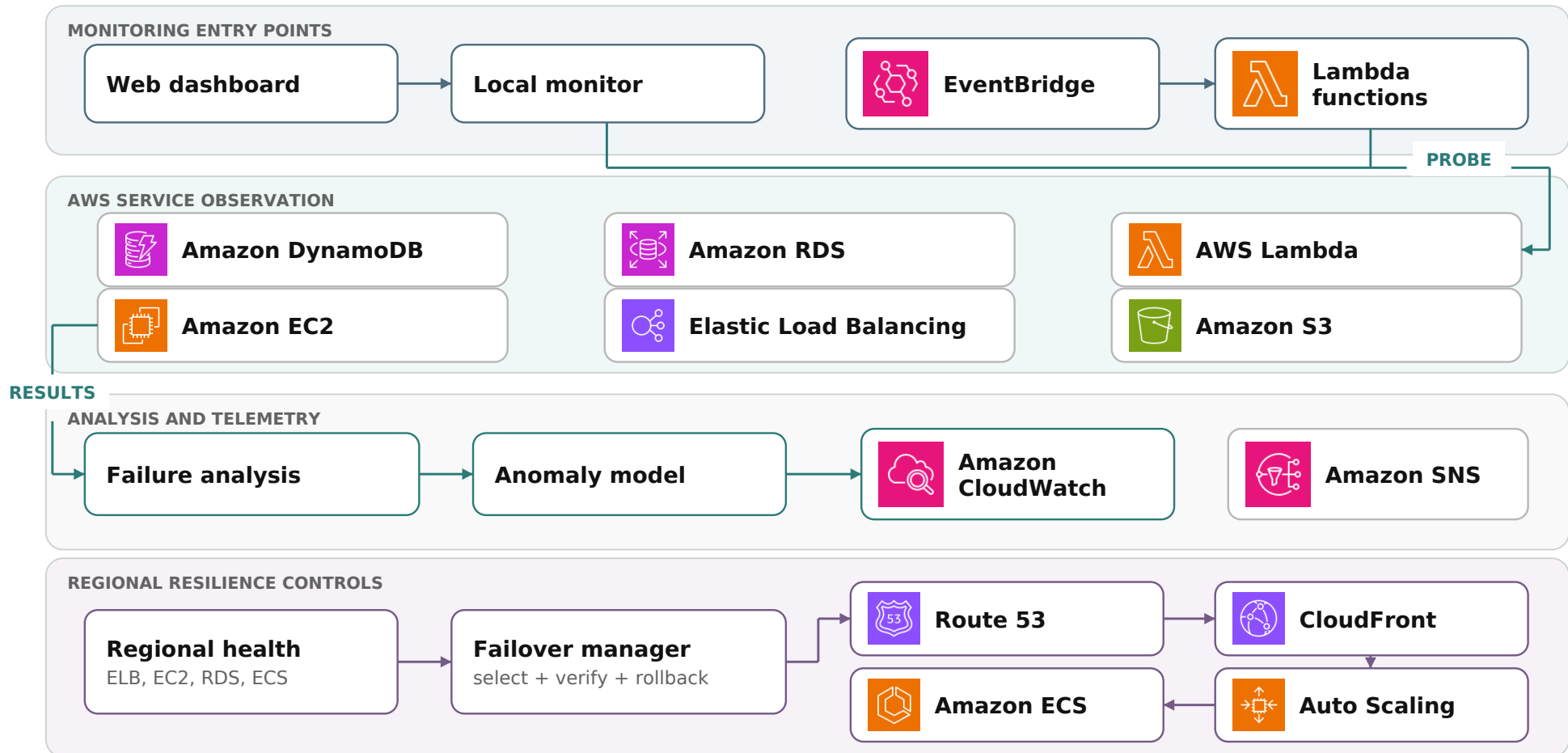


Figure 1: Monitoring, analysis, telemetry and multi-region failover boundaries.

3 Architecture interpretation

3.1 Monitoring paths

The local path begins with the Flask dashboard or the core monitoring process. Both resolve regional AWS endpoints directly; the core monitor adds TCP port 443 checks, retains recent health checks and invokes failure analysis. The AWS-side path begins with EventBridge rules that invoke regional Lambda functions. The monitoring function resolves a narrower endpoint set—DynamoDB, RDS, Lambda, EC2 and ELB—and publishes results to CloudWatch.

3.2 Observation targets

Endpoint names are constructed from service-specific regional templates. DynamoDB includes API and Streams endpoints. RDS, Lambda, EC2 and ELB use their regional control-plane endpoints. S3 is present in the core and dashboard monitors but is not included in the inline scheduled monitoring function.

3.3 Analysis and telemetry

Core checks feed rule-based failure aggregation, cascade-risk estimation and optional anomaly analysis. Both local and scheduled paths can write custom metrics to CloudWatch. The repository also creates CloudWatch dashboards and three alarm definitions. An SNS topic is provisioned for critical alerts; notification wiring remains an explicit integration step rather than an assumed path in the diagram.

3.4 Regional resilience controller

The failover manager queries ELB, EC2, RDS and ECS state through regional boto3 clients. It ranks healthy secondary regions, changes Route 53 records and CloudFront origins, raises Auto Scaling and ECS capacity, and verifies the target region before completing the event. If a failover stage or final verification fails, DNS and CloudFront changes are reversed.

4 DNS health collection

The core monitor uses dnspython to resolve A records with a five-second resolver timeout and ten-second lifetime. Each result records the endpoint, service, region, success state, response time, resolved addresses, timestamp and any error. When DNS succeeds, a second path attempts a TCP connection to port 443. The monitor retains the latest 1,000 checks in memory and runs its main loop every 30 seconds.

4.1 Default coverage

Service	Endpoint coverage	Default regions
DynamoDB	API and Streams	us-east-1, us-west-2, eu-west-1
RDS	Regional service endpoint	us-east-1, us-west-2, eu-west-1
Lambda	Regional service endpoint	us-east-1, us-west-2, eu-west-1
EC2	Regional service endpoint	us-east-1, us-west-2, eu-west-1
ELBv2	Regional service endpoint	us-east-1, us-west-2, eu-west-1
S3	Regional endpoint variants	us-east-1, us-west-2, eu-west-1

4.2 Status window

The current-status API summarizes checks from the preceding five minutes and alerts from the preceding thirty minutes. It returns service-level health, latest alerts, monitoring state and the configured region list. These windows are computed from in-memory records and reset when the process restarts.

4.3 Web monitor

The dashboard monitor performs direct hostname resolution with the Python socket library, updates status every ten seconds and broadcasts data through Socket.IO. The status endpoint is also available as JSON at `/api/status`. Historical dashboard summaries are held in memory, with the latest twenty entries returned from a fifty-entry buffer.

5 Detection and risk analysis

5.1 Failure aggregation

Checks are grouped by service and region. A DynamoDB DNS failure creates a critical alert because the service is treated as a high-impact dependency. Other services generate high or critical DNS-failure alerts when at least half of their checks fail. Each alert includes affected endpoints, failure rate, predicted impact, recommended actions and an optional cascade-risk record.

5.2 Cascade model

The dependency map expresses likely downstream exposure—for example, DynamoDB lists Lambda, ECS, EC2 and RDS; RDS lists Lambda, ECS and EC2. Cascade score combines failed endpoint count, dependent-service count and service criticality, then caps the result at 1.0. The output records affected services, estimated impact, mitigation actions and confidence.

```
cascade_score = min(1.0,
failed_endpoint_count * 0.3
+ dependent_service_count * 0.4
+ criticality
)
```

5.3 Statistical anomaly path

When enough history is available, the monitor standardizes success, response time and hour-of-day features and fits an Isolation Forest with five percent contamination and a fixed random seed. If more than ten percent of evaluated samples are marked anomalous, it emits a medium-severity DNS anomaly alert. This model describes unusual observations within the current batch; it is not a pre-trained forecasting service.

5.4 Features, training and interpretation

The online anomaly helper builds three numeric features per check: resolution success as 0 or 1, response time, and hour of day. Service and region are retained as metadata but are not model inputs. It standardizes the current feature matrix and calls `fit_predict` on an Isolation Forest configured with contamination 0.05 and `random_state` 42. The helper requires at least ten rows, while its caller invokes it only when at least twenty checks are available.

A prediction of -1 marks an observation as anomalous. The alert condition requires more than 10 percent of rows to be anomalous, which is stricter than the configured 5 percent contamination

setting and can make alerts uncommon. No labeled holdout evaluation, accuracy percentage or forecasting horizon is established by this implementation.

`train_model.py` separately reads a CSV, derives the same three features, fits a scaler and forest, and serializes both with `joblib`. Its command-line path creates a three-row demonstration dataset if the CSV is absent. The live monitor fits its own batch model; it does not load that saved artifact.

6 Multi-region health and failover

The failover manager defaults to `us-east-1` as primary, with `us-west-2` and `eu-west-1` as secondary regions. A region is considered ready when its aggregate service score is at least 0.7 and its health-check duration is below five seconds. The continuous manager checks every minute and tracks three consecutive failures before initiating selection, while also permitting an immediate trigger when the active health score falls below threshold.

6.1 Regional health inputs

Service	Health signal
Elastic Load Balancing	Load-balancer state and registered target health
Amazon EC2	Running instances and instance status checks
Amazon RDS	Database instances in the available state
Amazon ECS	Service state and running-task coverage

6.2 Failover transaction

- Select the healthy secondary region with the highest score, using response time as the tie-breaker.
- UPSERT Route 53 A and CNAME records whose values contain the source-region identifier.
- Update CloudFront origin domain names that contain the source-region identifier.
- Increase Auto Scaling desired capacity and ECS desired task count in the target region.
- Verify target health for up to five minutes; reverse Route 53 and CloudFront substitutions if the transaction does not complete.
- Record the failover event and active region in process memory.

7 Telemetry and operational views

7.1 Core metrics

Namespace / surface	Implemented outputs
Custom/DNSOutagePrevention	DNS success rate, DNS response time and overall DNS health, with service and region dimensions where applicable.
Scheduled monitor	Endpoint success, endpoint response time, DNS health score and regional health score.
CloudWatch dashboards	DNS health, endpoint latency, system health, cascade risk, anomaly score and AWS service metrics.
CloudWatch alarms	SystemHealthScore below 0.7, AnomalyScore above 0.8 and PredictedOutageRisk above 0.9.
Local dashboard	Multi-region overview, service health matrix, endpoint details, update stream and recent history.

7.2 Operator access

Running `launch_dashboard.py` starts the local web server on port 5000, starts its background monitor and opens the default browser. The underlying Flask application binds to all interfaces. Terminal-oriented monitor scripts provide additional direct status views and CloudWatch setup utilities without requiring the web UI.

7.3 State model

Health checks, alerts, regional health and failover events are maintained in process memory. CloudWatch provides durable metric history once metric publication succeeds, but application event records are not written to a project-owned database or object store.

8 Infrastructure and deployment

The Python deployer creates a Lambda execution role and policy, regional monitoring and health-check functions, EventBridge schedules, SNS topics and CloudWatch dashboards. The primary resilience CloudFormation template creates a VPC, three public subnets, two private subnets, an internet gateway, public routing, an internet-facing Application Load Balancer, its security group, an ECS cluster with Fargate capacity providers and a CloudWatch dashboard.

8.1 Scheduled functions

Function	Schedule	Responsibility
DNS monitoring	Every minute	Resolve five AWS service endpoint families and publish endpoint and regional DNS metrics.
Regional health check	Every five minutes	Instantiate the failover manager, evaluate configured regions and publish regional health scores.

8.2 Infrastructure boundary

The resilience template establishes network, load-balancing and cluster foundations. It does not define an application task definition, ECS service, target group or ALB listener. Those application resources belong in a deployment layer that supplies the workload being protected. The separate `cloudformation.yaml` describes a crypto-security EKS stack and is not part of the DNS resilience architecture represented here.

8.3 Runtime dependencies

The core requirements file declares boto3, NumPy, pandas, scikit-learn and dnspython. The web surface additionally imports Flask and Flask-SocketIO, and model serialization in `train_model.py` uses joblib.

9 Security and operating boundaries

9.1 AWS permissions

The deployer creates a role with CloudWatch and log access, service-description permissions, Auto Scaling updates, Route 53 and CloudFront changes, AWS Health and Support reads, and SNS publishing. These privileges enable the monitoring and failover operations described above. Production deployment should scope resources and actions to the hosted zones, distributions, clusters, services and scaling groups owned by this system.

9.2 Change controls

Route 53 and CloudFront changes are based on replacing a source-region string inside existing values. This is appropriate for a controlled demonstration where record and origin naming follows that convention. A production rollout should use explicit resource identifiers, a reviewed allowlist, pre-change snapshots and an operator approval or policy gate for traffic mutation.

9.3 Dashboard exposure

The development dashboard binds to `0.0.0.0:5000`, uses a static application secret and permits Socket.IO origins broadly. It should remain local or sit behind authentication and a reverse proxy until secrets, TLS, access control and origin policy are configured.

9.4 Cost and API behavior

Continuous multi-region checks, custom metrics, dashboards, alarms, Lambda invocations and routing updates can incur AWS charges and API throttling. Check frequency, metric cardinality and the number of scanned hosted zones or distributions should be bounded for the target account.

10 Implementation cross-reference

Source reference: [AWS-DNS-Outage](#), revision [ab42f8634640](#). The review covers source and configuration; it does not certify a live deployment or every runtime outcome.

Source	Primary responsibility
outage_prevention_system.py	Core endpoint checks, metric collection, failure grouping, cascade analysis, anomaly detection, alert records and CloudWatch publication.
multi_region_failover.py	Regional service health, target selection, Route 53 and CloudFront updates, target scaling, verification and rollback.
deployment_script.py	IAM, Lambda, EventBridge, SNS, CloudFormation and dashboard provisioning.
resilience_architecture.yaml	VPC, subnets, internet gateway, routing, ALB, security group, ECS cluster and dashboard foundations.
web_dashboard.py / dashboard.html	Local Flask and Socket.IO monitoring API and browser visualization.
live_dashboard.py	CloudWatch dashboard, metric and alarm creation.
simple_dns_monitor.py	Compact terminal monitoring and CloudWatch publication path.
enhanced_dns_monitor.py	Expanded terminal monitoring and CloudWatch resource management.
launch_dashboard.py	Dependency check and local dashboard startup.
train_model.py	Standalone Isolation Forest training and joblib serialization utility.
requirements.txt	Core Python dependency declarations.

10.1 Configuration points

Setting	Default / location
Regions	us-east-1, us-west-2, eu-west-1 in monitor and deployer constructors
Primary region	us-east-1 in MultiRegionFailoverManager
Health threshold	0.7
Response-time threshold	5.0 seconds
Consecutive failures	3
Core monitor interval	30 seconds
Web dashboard interval	10 seconds
Failover manager interval	60 seconds

11 Next steps and completion criteria

The implementation demonstrates the monitoring model, operational views and the complete failover transaction in code. The following work would convert those components into a reproducible production deployment while preserving the present design.

Priority	Next step	Completion criterion
1	Package deployment entry points	The deploy command imports repository modules from their actual paths and every Lambda artifact includes its Python modules and third-party dependencies.
2	Connect scheduled health to failover	The scheduled controller evaluates threshold state, invokes the reviewed failover transaction and persists its decision and event record.
3	Wire notifications	CloudWatch alarm actions publish to the intended SNS topics and subscriptions are verified in each deployment region.
4	Complete workload infrastructure	The protected ECS task definition, service, target group and ALB listeners are provisioned and pass health checks in each target region.
5	Constrain mutations	Route 53, CloudFront, Auto Scaling and ECS operations use explicit allowlisted identifiers, bounded capacity and least-privilege IAM.
6	Persist and test state	Checks, alerts and failover events survive restarts; automated tests cover DNS failure, target selection, successful verification and rollback.
7	Harden operator access	Dashboard authentication, managed secrets, TLS, origin restrictions and production server settings are enabled.
8	Reconcile project assets	The unrelated EKS template is archived or renamed, runtime dependencies are complete, and documented commands pass in a clean environment.

11.1 Delivery sequence

Package and test the two scheduled functions first, then connect alarm and failover decisions in a non-production AWS account. Complete the protected workload infrastructure before enabling traffic mutation. Finally, restrict IAM and resource selection, add persistent evidence, and expose the dashboard only after access controls are in place.